

UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN GENERAL DE PREGRADO
PROYECTO DE CARRERA: INGENIERÍA EN INFORMÁTICA
ESTRUCTURA DE DATOS

Proyecto #1

Juego Reversi en C++

Integrantes:	Keymers Campos CI: V-32.470.476 Sebastián Quintana CI: V-32.340.839 Christopher Lara CI: V-31.772.309 Alfredo Díaz CI:V-32.094.771
Profesor:	Carlos Abaffy
Asignatura:	Estructura de Datos
Fecha:	Ciudad Guayana, Mayo de 2026

Contenido

1. Introducción	3
2. Análisis del Problema	4
2.1 Contexto y Descripción del Juego	4
2.2 Requerimientos Funcionales	4
2.3 Requerimientos No Funcionales.....	4
2.4 Desafíos y Consideraciones de Diseño.....	4
3. Estructura de Datos y Diagrama de Clases	6
3.1 Representación del Tablero	6
3.2 Descripción de las Clases.....	6
3.3 Diagrama de Clases (UML Textual)	6
4. Descripción de Algoritmos	7
4.1 Algoritmo de Validación de Movimiento (Flanqueo).....	7
4.2 Algoritmo de la Computadora (Estrategia Greedy)	7
4.3 Algoritmo de Detección de Fin de Partida	8
4.4 Algoritmo de Guardado y Carga de Partida	8
5. Tabla de Funciones	10
6. Código Fuente	12
6.1 capaDeNegocio.h.....	12
6.2 capaDeNegocio.cpp.....	12
6.3 capaDePresentacion.h.....	14
6.4 capaDePresentacion.cpp.....	14
6.5 capaDeDatos.h y capaDeDatos.cpp	15
6.6 MAIN.cpp.....	16
7. Conclusión	17
7.1 Limitaciones Identificadas.....	17
7.2 Mejoras Propuestas	17

1. Introducción

El Reversi (también conocido como Othello) es un juego de tablero clásico para dos jugadores que combina estrategia, control de posición y capacidad de anticipación. En el marco de la asignatura Estructura de Datos, se ha desarrollado una implementación completa del juego en C++, siguiendo una arquitectura de tres capas (presentación, negocio y datos). El programa permite modalidades de juego entre humanos, contra la computadora y simulación PC vs PC, además de funcionalidades de guardado y carga de partidas.

El objetivo de este proyecto es aplicar conceptos fundamentales de la programación orientada a objetos, manejo de memoria dinámica, persistencia de datos y diseño de algoritmos eficientes. El sistema implementa la lógica completa del juego: inicialización del tablero, validación de movimientos mediante flanqueo en ocho direcciones, volteo de fichas, detección de fin de partida y cómputo del ganador.

A continuación se presenta el análisis completo del problema, la estructura de clases utilizada, los algoritmos implementados con su análisis de complejidad, el código fuente detallado y las conclusiones del proyecto.

2. Análisis del Problema

2.1 Contexto y Descripción del Juego

El Reversi se juega en un tablero de 8×8 casillas. Cada jugador dispone de fichas de un color (negras y blancas). Al inicio, se colocan cuatro fichas en el centro del tablero en posición alternada. El jugador con fichas negras siempre mueve primero.

Un movimiento es válido únicamente si la ficha colocada flanquea al menos una ficha del oponente en alguna de las ocho direcciones posibles (horizontal, vertical y diagonal), teniendo una ficha propia al otro extremo de la línea. Todas las fichas flanqueadas se voltean al color del jugador que movió. Si un jugador no tiene movimientos válidos, debe pasar su turno. El juego termina cuando ninguno de los dos jugadores puede realizar un movimiento válido. Gana el jugador con más fichas de su color en el tablero.

2.2 Requerimientos Funcionales

- Inicialización del tablero con las cuatro fichas centrales según las reglas oficiales.
- Validación de movimientos según las reglas de flanqueo en las ocho direcciones.
- Aplicación de volteos al realizar un movimiento válido.
- Detección de turnos sin movimientos disponibles y pase automático del turno.
- Finalización del juego cuando ningún jugador puede mover.
- Conteo final de fichas y determinación del ganador.
- Menú interactivo con cinco opciones: Jugador vs Jugador, Jugador vs PC (con selección de color), PC vs PC, Cargar partida guardada y Salir.
- Persistencia del estado del juego (tablero y turno) en un archivo de texto plano.
- Interfaz de consola que muestre el tablero con coordenadas A–H / 1–8 y el conteo de fichas en tiempo real.

2.3 Requerimientos No Funcionales

- Usabilidad: la entrada de movimientos es sencilla (ej. "D3") y se informa claramente sobre errores de entrada.
- Rendimiento: la IA evalúa todas las jugadas posibles en milisegundos; no existe demora perceptible para el usuario.
- Mantenibilidad: el código está organizado en capas bien diferenciadas para facilitar modificaciones futuras.
- Portabilidad: salvo la pausa con windows.h, el resto del código es C++ estándar.

2.4 Desafíos y Consideraciones de Diseño

- Manejo de las ocho direcciones de flanqueo con arreglos dx/dy para evitar código repetitivo.
- Eficiencia de la IA: evaluar hasta 64 movimientos con simulación de tablero debe ser rápido.
- Detección correcta del fin de partida cuando ninguno de los dos jugadores puede mover.
- Persistencia completa: guardar y restaurar tanto el tablero como el turno actual.
- Validación de entrada: convertir coordenadas alfanuméricas ("D3") a índices de matriz y manejar mayúsculas/minúsculas.
- Separación estricta de responsabilidades entre las tres capas del sistema.

3. Estructura de Datos y Diagrama de Clases

3.1 Representación del Tablero

El tablero se representa como un vector bidimensional de enteros (vector<vector<int>>) de 8×8. Cada celda puede tener tres valores:

- 0 — casilla vacía
- 1 — ficha negra
- 2 — ficha blanca

Esta representación permite acceso en $O(1)$ a cualquier celda y es compatible con operaciones de copia directa para la simulación de jugadas en la IA.

3.2 Descripción de las Clases

Clase capaDeNegocio

Contiene toda la lógica del juego: estado del tablero, reglas de movimiento, control de turnos, conteo de fichas e inteligencia artificial. Es el núcleo del sistema.

Atributo / Constante	Tipo	Descripción
tablero	vector<vector<int>>	Matriz 8×8: 0=vacío, 1=negras, 2=blancas
turnoActual	int	Jugador activo: 1 (negras) o 2 (blancas)
dx[8]	const int[]	Desplazamientos de fila para las 8 direcciones
dy[8]	const int[]	Desplazamientos de columna para las 8 direcciones

Clase capaDePresentacion

Gestiona la interacción con el usuario: muestra el tablero, controla el menú principal y coordina el flujo de la partida. Contiene instancias de las otras dos clases.

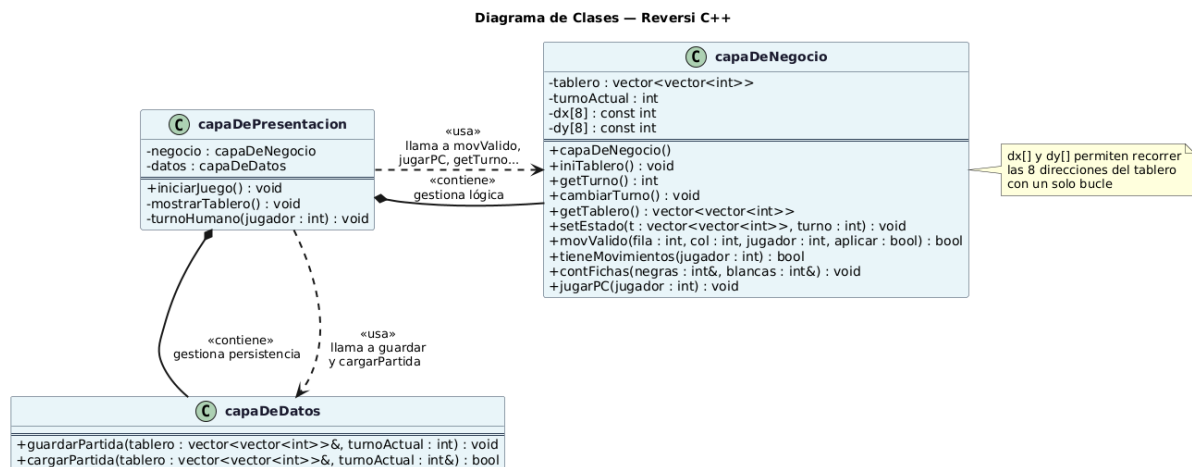
Atributo	Tipo	Descripción
negocio	capaDeNegocio	Objeto con toda la lógica del juego
datos	capaDeDatos	Objeto para guardar y cargar partidas

Clase capaDeDatos

Responsable de la persistencia: escribe y lee el estado del juego en un archivo de texto plano (reversi_guardado.txt). No contiene atributos de estado propios.

3.3 Diagrama de Clases

Las relaciones entre clases siguen el patrón de composición/uso:



4. Descripción de Algoritmos

4.1 Algoritmo de Validación de Movimiento (Flanqueo)

La función `movValido(fila, col, jugador, aplicar)` es el corazón del juego. Determina si colocar una ficha en (fila, col) es legal para el jugador indicado, y opcionalmente aplica el volteo.

Pseudocódigo

```

FUNCIÓN movValido(fila, col, jugador, aplicar):
    SI celda fuera de rango O celda ocupada → retornar FALSO
    oponente = contrario(jugador)
    jugadaValida = FALSO
    PARA CADA dirección dir en {N, NE, E, SE, S, SO, O, NO}:
        x = fila + dx[dir], y = col + dy[dir]
        MIENTRAS tablero[x][y] == oponente:
            avanzar en dirección dir
        SI se encontró al menos un oponente Y tablero[x][y] == jugador:
            jugadaValida = VERDADERO
            SI aplicar:
                voltear todas las fichas intermedias
    SI jugadaValida Y aplicar: colocar ficha en (fila,col)
    retornar jugadaValida
    
```

Análisis de Complejidad

- Bucle de 8 direcciones, cada una recorre como máximo 6 casillas intermedias.
- Total de operaciones: $8 \times 6 = 48 \rightarrow O(1)$ (el tablero es de tamaño fijo).
- Complejidad espacial: $O(1)$, solo variables locales.

4.2 Algoritmo de la Computadora (Estrategia Greedy)

La función `jugarPC(jugador)` implementa una estrategia de maximización inmediata de fichas (greedy). No anticipa turnos futuros; elige la jugada que deja más fichas propias tras el movimiento.

Pseudocódigo

```
FUNCIÓN jugarPC(jugador):
    mejorFila = -1, mejorCol = -1, maxFichas = -1
    PARA i = 0..7:
        PARA j = 0..7:
            SI movValido(i, j, jugador, aplicar=FALSO):
                tableroTemp = copia(tablero) // O(64)
                movValido(i, j, jugador, aplicar=VERDADERO) // aplica volteo
                fichas = contFichas(jugador) // O(64)
                SI fichas > maxFichas:
                    maxFichas = fichas
                    mejorFila = i, mejorCol = j
                tablero = tableroTemp // restaurar
    SI mejorFila != -1:
        movValido(mejorFila, mejorCol, jugador, aplicar=VERDADERO)
```

Análisis de Complejidad Detallado

Sea $n = 64$ el número total de celdas del tablero (8×8 fijo):

Operación	Costo unitario	Veces (peor caso)
Verificar movValido (sin aplicar)	$O(1) \approx 48$ ops	64 (todas las celdas)
Copia del tablero	64 asignaciones	≤ 64 celdas válidas
Aplicar movimiento (volteo)	≤ 56 volteos	≤ 64 celdas válidas
contFichas	64 accesos	≤ 64 celdas válidas
Restauración del tablero	64 asignaciones	≤ 64 celdas válidas
TOTAL estimado (peor caso)	≈ 250 ops/mov	$64 \times 250 \approx 16,000$ operaciones

Conclusión: dado que el tablero es de tamaño constante, la complejidad asintótica es $O(1)$. En la práctica, el algoritmo ejecuta alrededor de 16,000 operaciones simples por turno, lo que es prácticamente instantáneo en cualquier hardware moderno.

Nota: esta es una estrategia greedy miope. No considera posiciones estratégicas (esquinas, bordes) ni anticipa respuestas del oponente. Una mejora natural sería implementar Minimax con poda alfa-beta.

4.3 Algoritmo de Detección de Fin de Partida

Después de cada turno, el sistema verifica si el jugador activo tiene movimientos disponibles mediante tieneMovimientos. Si no los tiene, se pasa el turno. Si el siguiente tampoco puede mover, la partida termina.

```
FUNCIÓN tieneMovimientos(jugador):
    PARA i = 0..7:
        PARA j = 0..7:
            SI movValido(i, j, jugador, aplicar=FALSO) → retornar VERDADERO
    retornar FALSO // ningún movimiento válido encontrado
```

Complejidad: $O(64 \times 48) = O(1)$ en tablero fijo. En el peor caso, 3,072 operaciones simples.

4.4 Algoritmo de Guardado y Carga de Partida

La persistencia se realiza sobre el archivo `reversi_guardado.txt`. El formato es simple: primera línea con el turno actual (1 o 2), seguido de 8 líneas con 8 enteros separados por espacio representando el tablero.

```
GUARDAR: escribir turnoActual → escribir tablero[0..7][0..7]
CARGAR:  leer turnoActual     → leer tablero[0..7][0..7]
Complejidad:  $O(64)$  lectura/escritura. Archivo  $\approx$  150 bytes.
```

5. Tabla de Funciones

Clase	Función	Descripción	Parámetros	Retorno
capaDeNegocio	capaDeNegocio()	Constructor: inicializa tablero y llama a iniTablero.	—	—
	iniTablero()	Coloca las 4 fichas iniciales y fija turno = 1.	void	void
	getTurno()	Retorna el turno activo (1: negras, 2: blancas).	void	int
	cambiarTurno()	Alterna el turno entre 1 y 2.	void	void
	getTablero()	Retorna una copia del tablero actual.	void	vector<vector<int>>
	setEstado()	Restaura tablero y turno desde datos externos.	vector<...> t, int turno	void
	movValido()	Verifica si un movimiento es legal; si aplicar=true, voltea fichas y coloca la nueva.	int fila, col, jugador; bool aplicar	bool
	tieneMovimientos()	Retorna true si el jugador tiene al menos un movimiento válido.	int jugador	bool
capaDePresentacion	contFichas()	Cuenta fichas negras y blancas en el tablero.	int& negras, int& blancas	void (por ref.)
	jugarPC()	Evalúa todas las jugadas posibles y ejecuta la que maximiza fichas propias.	int jugador	void
	iniciarJuego()	Bucle principal: muestra menú, gestiona partidas y turnos.	void	void
	mostrarTablero()	Imprime el tablero en consola con coordenadas y conteo de fichas.	void	void
	turnoHumano()	Lee la jugada del usuario, valida el formato y aplica el movimiento. Permite guardar con 'G'.	int jugador	void

capaDeDatos	guardarPartida()	Escribe tablero y turno en reversi_guardado.txt.	const vector<...>& tablero, int turno	void
	cargarPartida()	Lee el archivo y restaura el estado. Retorna true si tuvo éxito.	vector<...>& tablero, int& turno	bool

6. Código Fuente

A continuación se lista el código fuente de cada archivo. Se respeta la arquitectura por capas y se incluyen comentarios explicativos en las secciones clave.

6.1 capaDeNegocio.h

```
#ifndef CAPA_DE_NEGOCIO_H
#define CAPA_DE_NEGOCIO_H

#include <vector>
using namespace std;

class capaDeNegocio {
private:
    vector<vector<int>> tablero;    // 8x8: 0=vacío, 1=negras, 2=blancas
    int turnoActual;              // 1=negras, 2=blancas

    // Vectores de dirección para las 8 direcciones posibles:
    // N, NE, E, SE, S, SO, O, NO
    const int dx[8] = {-1, -1, -1, 0, 0, 1, 1, 1};
    const int dy[8] = {-1, 0, 1, -1, 1, -1, 0, 1};

public:
    capaDeNegocio();
    void iniTablero();
    int getTurno();
    void cambiarTurno();
    vector<vector<int>> getTablero();
    void setEstado(vector<vector<int>> t, int turno);
    bool movValido(int fila, int col, int jugador, bool aplicar = false);
    bool tieneMovimientos(int jugador);
    void contFichas(int& negras, int& blancas);
    void jugarPC(int jugador);
};

#endif
```

6.2 capaDeNegocio.cpp

```
#include "capaDeNegocio.h"
#include <iostream>
using namespace std;

// Constructor: crea el tablero vacío y lo inicializa
capaDeNegocio::capaDeNegocio() {
    tablero = vector<vector<int>>(8, vector<int>(8, 0));
    iniTablero();
}

// Posición inicial: 4 fichas en el centro, negras inician
void capaDeNegocio::iniTablero() {
    for(int i = 0; i < 8; i++) fill(tablero[i].begin(), tablero[i].end(), 0);
    tablero[3][3] = 2; tablero[4][4] = 2;    // Blancas
    tablero[3][4] = 1; tablero[4][3] = 1;    // Negras
    turnoActual = 1;
}

int capaDeNegocio::getTurno()    { return turnoActual; }
void capaDeNegocio::cambiarTurno() { turnoActual = (turnoActual == 1) ? 2 : 1; }
}
```

```

vector<vector<int>>> capaDeNegocio::getTablero() { return tablero; }
void capaDeNegocio::setEstado(vector<vector<int>>> t, int turno) {
    tablero = t; turnoActual = turno;
}

// Verifica si colocar en (fila,col) es válido para jugador.
// Si aplicar=true, ejecuta el volteo y coloca la ficha.
bool capaDeNegocio::movValido(int fila, int col, int jugador, bool aplicar) {
    if (fila<0||fila>=8||col<0||col>=8||tablero[fila][col]!=0) return false;
    bool jugadaValida = false;
    int oponente = (jugador == 1) ? 2 : 1;
    for (int dir = 0; dir < 8; dir++) {
        int x = fila + dx[dir], y = col + dy[dir];
        bool encontroOponente = false;
        // Avanzar mientras haya fichas del oponente en esta dirección
        while (x>=0&&x<8&&y>=0&&y<8&&tablero[x][y]==opponente) {
            encontroOponente = true;
            x += dx[dir]; y += dy[dir];
        }
        // Si encontramos al menos un oponente y luego una ficha propia: válido
        if (encontroOponente&&x>=0&&x<8&&y>=0&&y<8&&tablero[x][y]==jugador) {
            jugadaValida = true;
            if (aplicar) { // Voltear fichas intermedias
                int vx = fila+dx[dir], vy = col+dy[dir];
                while (vx != x || vy != y) {
                    tablero[vx][vy] = jugador;
                    vx += dx[dir]; vy += dy[dir];
                }
            }
        }
    }
    if (jugadaValida && aplicar) tablero[fila][col] = jugador;
    return jugadaValida;
}

// Retorna true si el jugador tiene al menos un movimiento válido
bool capaDeNegocio::tieneMovimientos(int jugador) {
    for (int i = 0; i < 8; i++)
        for (int j = 0; j < 8; j++)
            if (movValido(i, j, jugador)) return true;
    return false;
}

// Cuenta fichas de cada color en el tablero
void capaDeNegocio::contFichas(int& negras, int& blancas) {
    negras = 0; blancas = 0;
    for (int i=0;i<8;i++) for (int j=0;j<8;j++)
        if (tablero[i][j]==1) negras++;
        else if (tablero[i][j]==2) blancas++;
}

// IA greedy: elige la jugada que maximiza las fichas propias
void capaDeNegocio::jugarPC(int jugador) {
    int mejorFila=-1, mejorCol=-1, maxVolteadas=-1;
    for (int i = 0; i < 8; i++) {
        for (int j = 0; j < 8; j++) {
            if (movValido(i, j, jugador)) {
                vector<vector<int>>> tableroTemp = tablero; // Guardar estado
                movValido(i, j, jugador, true); // Simular jugada
                int nN, nB; contFichas(nN, nB);
                int ganancia = (jugador==1) ? nN : nB;
                if (ganancia > maxVolteadas) {
                    maxVolteadas = ganancia;
                    mejorFila = i; mejorCol = j;
                }
            }
        }
        tablero = tableroTemp; // Restaurar estado
    }
}

```

```

    }
}
if (mejorFila != -1) {
    movValido(mejorFila, mejorCol, jugador, true); // Ejecutar mejor jugada
    cout << "La PC jugo en: " << char('A'+mejorCol) << mejorFila+1 << "\n";
}
}
}

```

6.3 capaDePresentacion.h

```

#ifndef CAPA_DE_PRESENTACION_H
#define CAPA_DE_PRESENTACION_H

#include "capaDeNegocio.h"
#include "capaDeDatos.h"

class capaDePresentacion {
private:
    capaDeNegocio negocio; // Lógica del juego
    capaDeDatos datos;     // Persistencia

    void mostrarTablero(); // Dibuja el tablero en consola
    void turnoHumano(int jugador); // Gestiona la entrada del usuario

public:
    void iniciarJuego(); // Punto de entrada principal
};

#endif

```

6.4 capaDePresentacion.cpp

```

#include "capaDePresentacion.h"
#include <iostream>
#include <string>
#include <windows.h> // Para Sleep(ms)
using namespace std;

// Muestra el tablero con coordenadas A-H / 1-8 y conteo de fichas
void capaDePresentacion::mostrarTablero() {
    vector<vector<int>> t = negocio.getTablero();
    cout << "\n  A B C D E F G H\n";
    for (int i = 0; i < 8; i++) {
        cout << i+1 << " ";
        for (int j = 0; j < 8; j++) {
            if (t[i][j]==0) cout << ". ";
            else if (t[i][j]==1) cout << "N "; // Negras
            else cout << "B "; // Blancas
        }
        cout << " " << i+1 << "\n";
    }
    cout << "  A B C D E F G H\n\n";
    int n, b; negocio.contFichas(n, b);
    cout << "Fichas -> Negras (N): " << n << " | Blancas (B): " << b << "\n";
}

// Lee movimiento del usuario, valida formato y aplica la jugada
void capaDePresentacion::turnoHumano(int jugador) {
    string entrada; bool jugadaHecha = false;
    while (!jugadaHecha) {
        cout << "Jugador " << (jugador==1?"Negras (N)":"Blancas (B)")
              << ", ingresa tu jugada (Ej: D3) o 'G' para guardar: ";
    }
}

```

```

        cin >> entrada;
        if (entrada=="G"||entrada=="g") {
            datos.guardarPartida(negocio.getTablero(), negocio.getTurno());
            continue;
        }
        if (entrada.length() < 2) { cout << "Entrada invalida.\n"; continue; }
        int col = toupper(entrada[0]) - 'A'; // 'A'→0 .. 'H'→7
        int fila = entrada[1] - '1'; // '1'→0 .. '8'→7
        if (negocio.movValido(fila, col, jugador, true)) jugadaHecha = true;
        else cout << "Movimiento invalido. Debes flanquear al menos una ficha
rival.\n";
    }
}

// Bucle principal: menú → partida → resultado → menú
void capaDePresentacion::iniciarJuego() {
    bool ejecutando = true;
    while (ejecutando) {
        int opcion;
        cout << "\n===== \n";
        cout << "=== B I E N V E N I D O   A R E V E R S I === \n";
        cout << "===== \n";
        cout << "1. Jugador vs Jugador\n2. Jugador vs PC\n";
        cout << "3. PC vs PC\n4. Cargar Partida Guardada\n5. Salir\n";
        cout << "Elige una opcion: "; cin >> opcion;
        if (opcion==5) { cout << "Saliendo...\n"; break; }
        // ... (lógica de modos y bucle de partida)
    }
}

```

6.5 capaDeDatos.h y capaDeDatos.cpp

```

// capaDeDatos.h
#ifndef CAPA_DE_DATOS_H
#define CAPA_DE_DATOS_H
#include <vector>
using namespace std;
class capaDeDatos {
public:
    void guardarPartida(const vector<vector<int>>& tablero, int turnoActual);
    bool cargarPartida(vector<vector<int>>& tablero, int& turnoActual);
};
#endif

// capaDeDatos.cpp
#include "capaDeDatos.h"
#include <iostream>
#include <fstream>
using namespace std;

// Escribe turno y tablero en reversi_guardado.txt
void capaDeDatos::guardarPartida(const vector<vector<int>>& tablero, int
turnoActual) {
    ofstream archivo("reversi_guardado.txt");
    if (archivo.is_open()) {
        archivo << turnoActual << endl; // Primera línea: turno
        for (int i=0; i<8; i++) {
            for (int j=0; j<8; j++) archivo << tablero[i][j] << " ";
            archivo << endl;
        }
        archivo.close();
        cout << "[SISTEMA] Partida guardada en reversi_guardado.txt\n";
    }
}

```

```
// Lee turno y tablero desde reversi_guardado.txt
bool capaDeDatos::cargarPartida(vector<vector<int>>& tablero, int& turnoActual)
{
    ifstream archivo("reversi_guardado.txt");
    if (archivo.is_open()) {
        archivo >> turnoActual;
        for (int i=0;i<8;i++) for (int j=0;j<8;j++) archivo >> tablero[i][j];
        archivo.close();
        return true;
    }
    return false; // Archivo no encontrado
}
```

6.6 MAIN.cpp

```
#include "capaDePresentacion.h"

// Punto de entrada: crea la capa de presentación e inicia el juego
int main() {
    capaDePresentacion juego;
    juego.iniciarJuego();
    return 0;
}
```


7. Conclusión

El desarrollo del Reversi ha permitido aplicar de manera integral los conceptos de programación orientada a objetos y estructura de datos estudiados en la asignatura. La separación en tres capas (presentación, negocio y datos) facilitó la organización del código y la asignación clara de responsabilidades a cada componente del sistema.

Se logró implementar completamente el juego con todas las reglas oficiales, ofreciendo modos de juego flexibles (humano vs humano, humano vs PC, PC vs PC) y funcionalidad de persistencia mediante archivos de texto. La estructura `vector<vector<int>>` resultó apropiada para representar el tablero, permitiendo tanto acceso $O(1)$ como copia eficiente para la simulación de jugadas de la IA.

El análisis de complejidad del algoritmo de la computadora confirma que, aunque evalúa hasta 16,000 operaciones por turno, la ejecución es prácticamente instantánea al tratarse de un tablero de tamaño fijo. Esto garantiza una experiencia de juego fluida incluso en hardware modesto.

7.1 Limitaciones Identificadas

- La capa de presentación accede directamente a la capa de datos, lo que viola el patrón de arquitectura por capas estricto. Lo correcto sería que la capa de datos fuera accedida únicamente desde la capa de negocio.
- Al cargar una partida guardada, se pierde el modo de juego original y se fuerza el modo Jugador vs Jugador.
- La IA es miope: maximiza fichas inmediatas sin considerar posiciones estratégicas (esquinas, bordes) ni anticipar respuestas del oponente.
- El uso de `windows.h` para la pausa limita la portabilidad a sistemas Windows.

7.2 Mejoras Propuestas

- Implementar una IA más avanzada con algoritmo Minimax y poda alfa-beta para anticipar varios turnos.
- Agregar una heurística posicional que valore las esquinas (+10) y bordes (+3) sobre las casillas centrales.
- Corregir la arquitectura para que la capa de presentación no acceda directamente a la capa de datos.
- Guardar también el modo de juego en el archivo de persistencia para restaurarlo correctamente.
- Reemplazar `windows.h` con `std::this_thread::sleep_for` de `<chrono>` para portabilidad multiplataforma.